

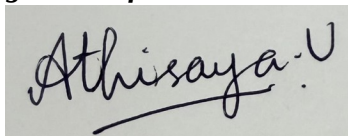
SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1202	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	Athisaya U	Reg.No.:	192571001
Department:	B.Tech CSE and Bioscience	Date:	7/8/26

Code of Conduct:

I Athisaya U / 192571001 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink that reads "Athisaya U". The signature is written in a cursive style with a horizontal line underneath the name.

ANNEXURE A

QUESTION 1

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

ANSWER

AIM

To design and simulate an 8-bit register-transfer system that performs **addition, subtraction, AND and OR** operations using multiple registers and an ALU.

COMPONENTS REQUIRED

Logisim Evolution, 8-bit registers R1, R2, R3 and ROUT, adder, subtractor, AND gate, OR gate, 4-to-1 multiplexer, clock and control pins.

PROCEDURE AND REGISTER-TRANSFER SEQUENCE

The 8-bit input values **05** and **03** are loaded into R1 and R2. Their outputs feed the arithmetic and logic circuits, and a 4-to-1 multiplexer selects one ALU result. On the active clock edge, the selected result is transferred into R3.

Clock / Select	Register-transfer operation	R3
T0	$R1 \leftarrow 05; R2 \leftarrow 03$	Operands loaded
T1 / 00	$R3 \leftarrow R1 + R2$	08
T2 / 01	$R3 \leftarrow R1 - R2$	02
T3 / 10	$R3 \leftarrow R1 \text{ AND } R2$	01
T4 / 11	$R3 \leftarrow R1 \text{ OR } R2$	07

The ALU performs the selected arithmetic or bitwise operation, while the multiplexer converts the two-bit ALU_SELECT control signal into the required output path. R3 stores the verified ALU result. ROUT remains 00 in these captures because its load control is disabled; this does not affect the result observed at R3.

SIMULATION OUTPUT

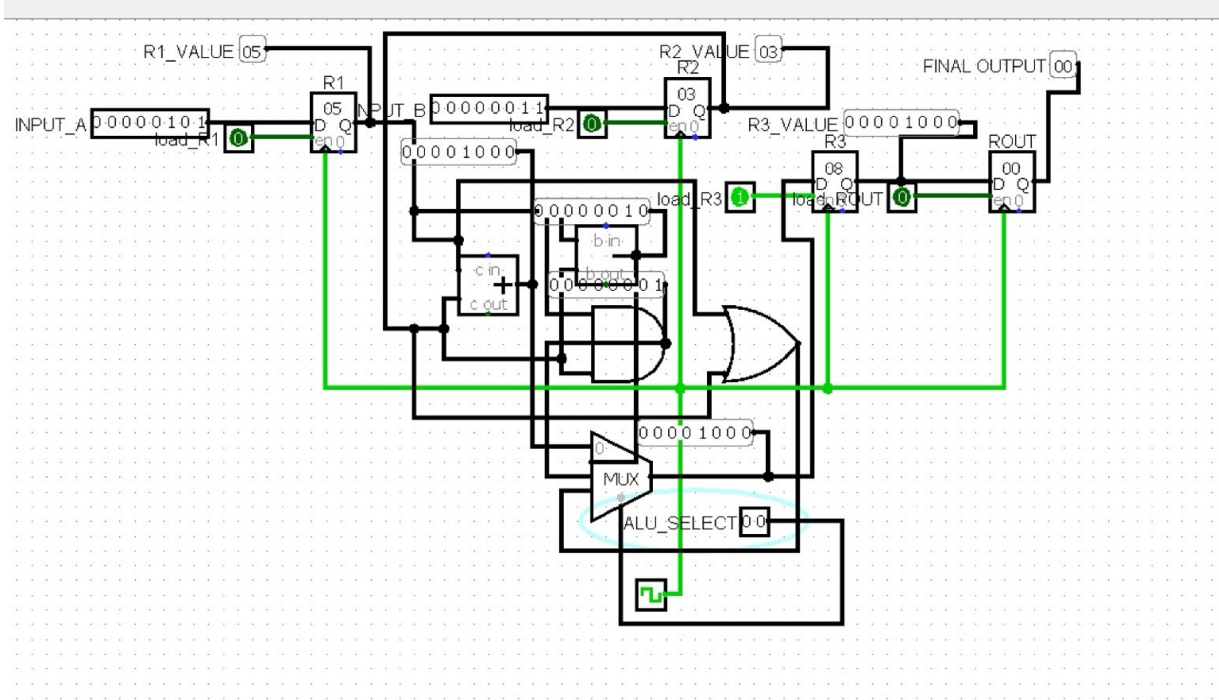


Figure 1(a): Addition selected (00); $05 + 03 = 08$.

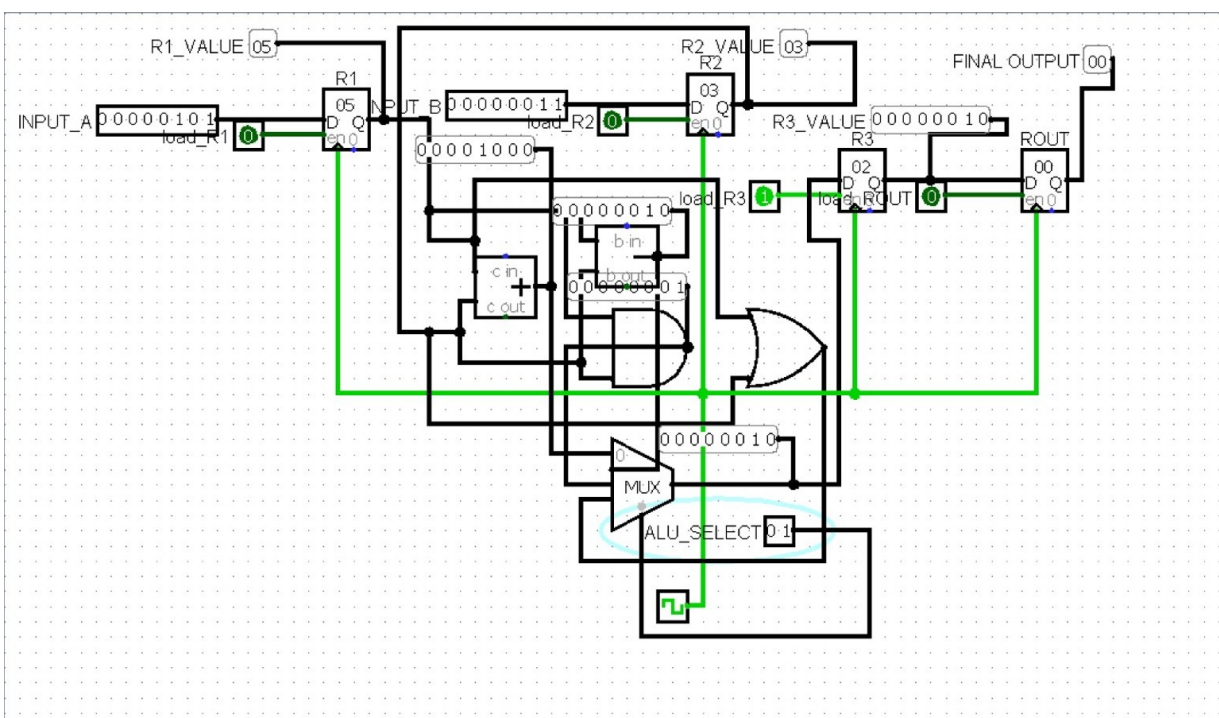


Figure 1(b): Subtraction selected (01); $05 - 03 = 02$.

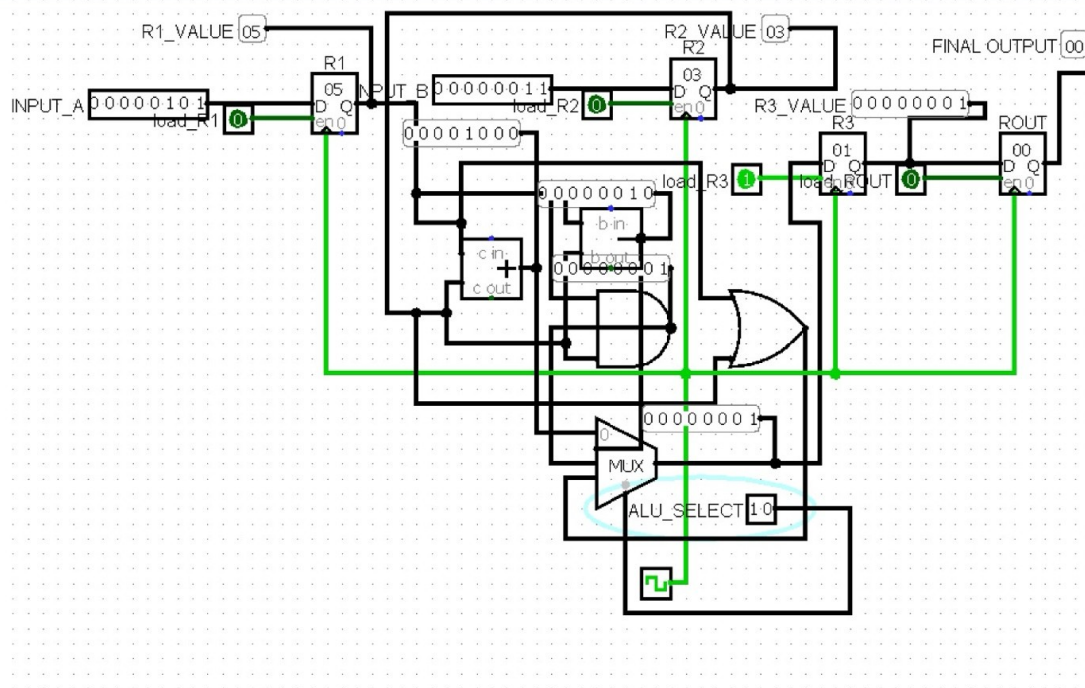


Figure 1(c): AND selected (10); $05 \text{ AND } 03 = 01$.

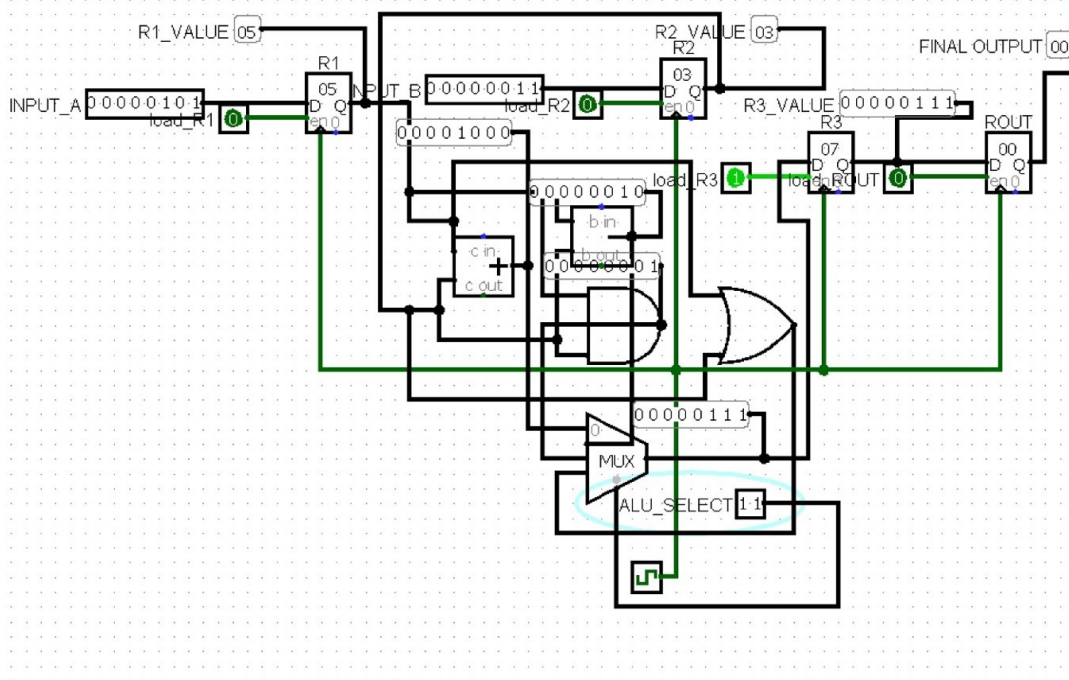


Figure 1(d): OR selected (11); $05 \text{ OR } 03 = 07$.

RESULT: All four ALU operations produced the correct 8-bit values in R3, demonstrating controlled register transfer and ALU processing.

QUESTION 2

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

ANSWER

AIM

To model **single-bus and multi-bus** computer organizations in Logisim, perform register, memory and I/O transfers, and compare their execution time and efficiency.

COMPONENTS REQUIRED

Logisim Evolution, 8-bit RAM, registers R1, R2, R3 and OUT, multiplexers, input pins, select/load controls, probes and a clock.

PROCEDURE

In the single-bus organization, RAM, input and registers share one data path. A multiplexer enables only one source at a time, so transfers must be scheduled sequentially. In the multi-bus organization, independent Bus A and Bus B paths allow two compatible transfers to occur during the same clock period.

SIMULATION OUTPUT

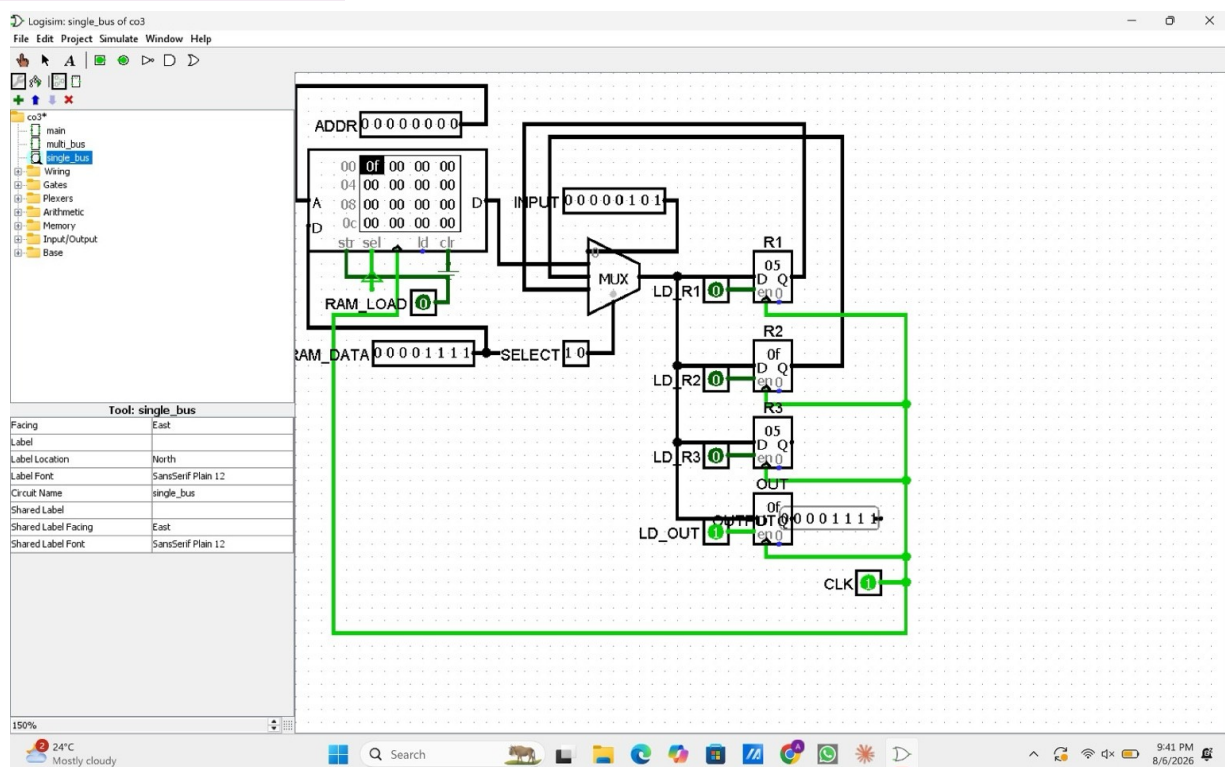


Figure 2(a): Single-bus Logisim model and observed register values.

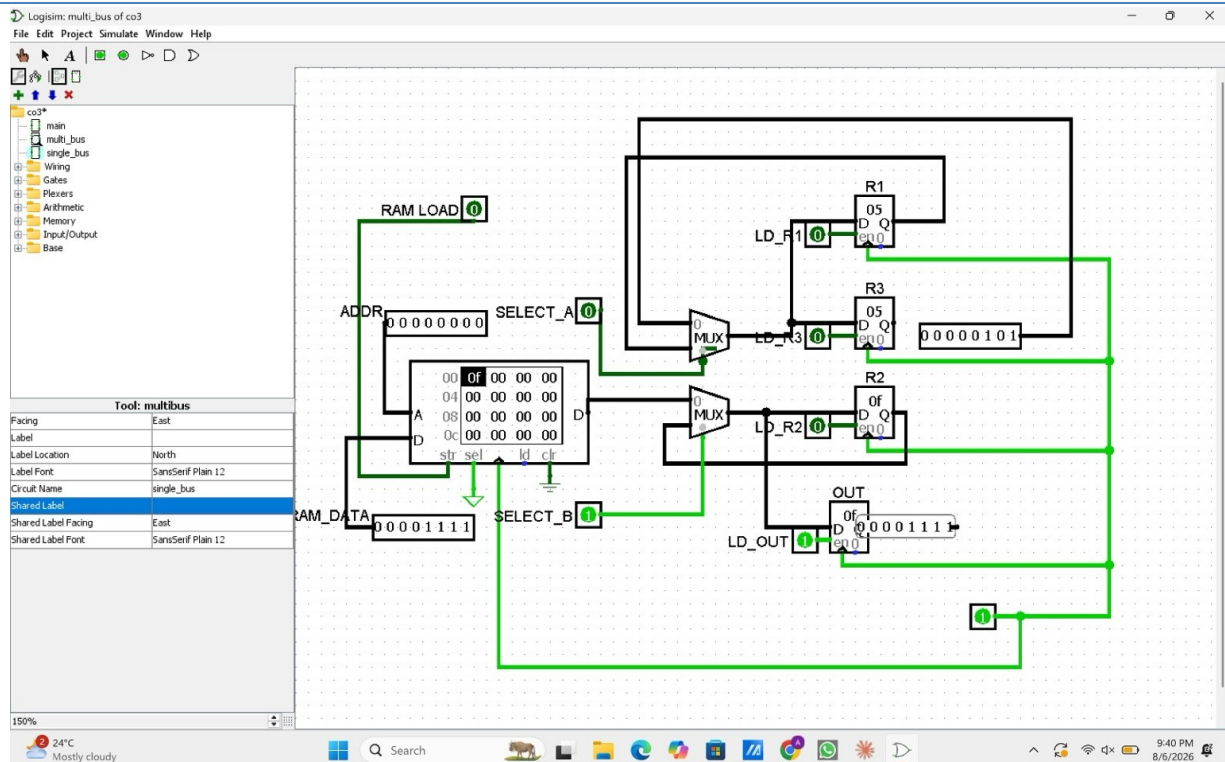


Figure 2(b): Multi-bus Logisim model with two independent transfer paths.

OBSERVATION AND ANALYSIS

Feature	Single bus	Multi bus
Data paths	One shared bus	Two independent buses
Simultaneous transfers	One	Up to two compatible transfers
Illustrative sequence	Input->R1, R1->R3, RAM->R2, R2->OUT	Input->R1 with RAM->R2; R1->R3 with R2->OUT
Clock cycles	4	2
Control complexity	Lower	Higher
Bottleneck	Frequent bus contention	Reduced contention

For one-cycle register transfers, time reduction = $(4 - 2) / 4 \times 100 = 50\%$, and ideal speedup = $4 / 2 = 2x$. The improvement occurs because the multi-bus datapath overlaps transfers that would otherwise compete for the single shared bus.

RESULT: For the demonstrated four transfers, the multi-bus model reduces the illustrative execution time from 4 cycles to 2 cycles and removes the principal shared-bus bottleneck.

QUESTION 3

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

ANSWER

AIM

To simulate a **five-stage instruction pipeline** and calculate its **speedup and throughput** relative to a non-pipelined processor.

ALGORITHM / PROCEDURE

The Python simulation moves five instructions through IF (Fetch), ID (Decode), EX (Execute), MEM (Memory) and WB (Write-back). After the pipeline begins filling, different instructions occupy different stages during the same cycle, increasing instruction completion rate.

PROGRAM AND OUTPUT

```

co3_ca_3.py - C:/Users/jimin/AppData/Local/Programs/Python/Python313/co3_ca_3.py (3.13.7)
File Edit Format Run Options Window Help

instructions = [
    "I1: ADD R1, R2, R3",
    "I2: SUB R4, R1, R5",
    "I3: AND R6, R4, R7",
    "I4: OR R8, R6, R9",
    "I5: ADD R10, R0, R11"
]

stages = ["IF", "ID", "EX", "MEM", "WB"]

number_of_instructions = len(instructions)
number_of_stages = len(stages)

pipeline_cycles = number_of_instructions + number_of_stages - 1
non_pipeline_cycles = number_of_instructions * number_of_stages

pipeline_table = []

for instruction_number in range(number_of_instructions):
    row = ["-" for cycle in range(pipeline_cycles)]

    for stage_number in range(number_of_stages):
        cycle_number = instruction_number + stage_number
        row[cycle_number] = stages[stage_number]

    pipeline_table.append(row)

print("FIVE-STAGE PIPELINE EXECUTION\n")
print("Instruction", end="\t")

for cycle in range(1, pipeline_cycles + 1):
    print("C" + str(cycle), end="\t")

print()

for i in range(number_of_instructions):
    print("I" + str(i + 1), end="\t\t")

    for value in pipeline_table[i]:
        print(value, end="\t")

    print()

speedup = non_pipeline_cycles / pipeline_cycles
throughput = number_of_instructions / pipeline_cycles

print("\nNon-pipelined cycles =", non_pipeline_cycles)
print("Pipelined cycles =", pipeline_cycles)
print("Speedup =", round(speedup, 2))
print("Throughput =", round(throughput, 2), "instructions per cycle")

```

Figure 3(a): Python program for five-stage pipeline scheduling.

```

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/jimin/AppData/Local/Programs/Python/Python313/co3_ca_3.py =
FIVE-STAGE PIPELINE EXECUTION

Instruction   C1    C2    C3    C4    C5    C6    C7    C8    C9
I1            IF    ID    EX    MEM   WB    -    -    -    -
I2            -    IF    ID    EX    MEM   WB    -    -    -
I3            -    -    IF    ID    EX    MEM   WB    -    -
I4            -    -    -    IF    ID    EX    MEM   WB    -
I5            -    -    -    -    IF    ID    EX    MEM   WB

Non-pipelined cycles = 25
Pipelined cycles = 9
Speedup = 2.78
Throughput = 0.56 instructions per cycle

>>>

```

Figure 3(b): Pipeline timing table and measured performance.

PERFORMANCE ANALYSIS

Measure	Formula	Verified value
Non-pipelined cycles	$n \times k$	$5 \times 5 = 25$
Pipelined cycles	$n + k - 1$	$5 + 5 - 1 = 9$
Speedup	$25 / 9$	$2.78x$
Throughput	$5 / 9$	$0.56 \text{ instructions/cycle}$

The simulation represents an ideal pipeline without stalls. The first instruction requires five stages, but later instructions overlap with it. Consequently, all five instructions complete in nine cycles rather than twenty-five; for a longer instruction stream, steady-state throughput approaches one completed instruction per cycle.

RESULT: Pipelining provides a measured speedup of 2.78x and throughput of 0.56 instructions per cycle for the five-instruction test sequence.

QUESTION 4

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

ANSWER

AIM

To simulate **data, control and structural hazards** and evaluate **stalling, forwarding and branch prediction** as hazard-resolution techniques.

HAZARD SCENARIO AND PROCEDURE

Hazard	Observed cause	Resolution
Data hazard	I2 reads R1 produced by I1 (RAW dependency)	Forward R1 to the dependent stage; stall only if required
Control hazard	I3 is BEQ and may change the next PC	Predict the branch; flush only on a wrong prediction
Structural hazard	Instruction fetch and LOAD may request one memory	Stall one request or provide separate instruction/data paths

PROGRAM AND OUTPUT

```

*co3_ca_3.py - C:/Users/jimin/AppData/Local/Programs/Python/Python313/co3_ca_3.py (3.13.7)
File Edit Format Run Options Window Help
instructions = [
    "I1: ADD R1, R2, R3",
    "I2: SUB R4, R1, R5",
    "I3: BEQ R4, R6, LABEL",
    "I4: AND R7, R8, R9",
    "I5: LOAD R10, [100]"
]

print("PIPELINE HAZARD SIMULATION\n")
print("Instructions:")

for instruction in instructions:
    print(instruction)

print("\nHazards Detected:")

print("\n1. Data Hazard:")
print("I2 depends on R1 produced by I1.")

print("\n2. Control Hazard:")
print("I3 is a branch instruction.")

print("\n3. Structural Hazard:")
print("Instruction fetch and memory access may require memory")
print("during the same cycle.")

without_handling_cycles = 14
without_handling_stalls = 5

with_stalling_cycles = 13
with_stalling_stalls = 4

with_forwarding_cycles = 10
with_forwarding_stalls = 1

with_prediction_cycles = 9
with_prediction_stalls = 0

print("\nPERFORMANCE COMPARISON")
print("Without Handling\t", without_handling_cycles, "\t", without_handling_stalls)
print("With Stalling\t\t", with_stalling_cycles, "\t", with_stalling_stalls)
print("With Forwarding\t\t", with_forwarding_cycles, "\t", with_forwarding_stalls)
print("Forwarding + Prediction\t", with_prediction_cycles, "\t", with_prediction_stalls)

improvement = (
    (without_handling_cycles - with_prediction_cycles)
    / without_handling_cycles
) * 100

print("\nPerformance Improvement =", round(improvement, 2), "%")

```

Figure 4(a): Python program modelling hazards and their handling methods.

```

===== RESTART: C:/Users/jimin/AppData/Local/Programs/Python/Python313/co3_ca_3.py =====
PIPELINE HAZARD SIMULATION

Instructions:
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: BEQ R4, R6, LABEL
I4: AND R7, R8, R9
I5: LOAD R10, [100]

Hazards Detected:

1. Data Hazard:
I2 depends on R1 produced by I1.

2. Control Hazard:
I3 is a branch instruction.

3. Structural Hazard:
Instruction fetch and memory access may require memory
during the same cycle.

PERFORMANCE COMPARISON

Method          Cycles  Stalls
Without Handling    14      5
With Stalling      13      4
With Forwarding    10      1
Forwarding + Prediction  9      0

Performance Improvement = 35.71 %

```

Figure 4(b): Detected hazards and performance comparison.

PERFORMANCE ANALYSIS

Method	Cycles	Stalls
Without handling	14	5
With stalling	13	4
With forwarding	10	1
Forwarding + prediction	9	0

Performance improvement = $(14 - 9) / 14 \times 100 = 35.71\%$. Stalling preserves correctness but loses cycles. Forwarding supplies a result before register write-back, while branch prediction avoids waiting for every branch decision.

RESULT: Combining forwarding and branch prediction reduces execution from 14 cycles with 5 stalls to 9 cycles with no stalls in the simulated scenario.

QUESTION 5

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

ANSWER

AIM

To analyze **superscalar, out-of-order, speculative and simultaneous multithreading** execution and compare **instruction throughput and resource utilization**.

PROCEDURE

A scalar processor issues one instruction per cycle. A superscalar processor has multiple execution units and can issue independent instructions together. Out-of-order execution selects ready instructions instead of waiting for program-order blockage. Speculative execution predicts control flow and temporarily executes the predicted path, committing results only when the prediction is validated. Hyper-threading is a practical form of SMT in which multiple hardware thread contexts share the same execution resources.

PROGRAM AND OUTPUT

```

co3_ca_3.py - C:/Users/jimin/AppData/Local/Programs/Python/Python313/co3_ca_3.py (3.13.7)
File Edit Format Run Options Window Help
instructions = [
    "ADD R1, R2, R3",
    "MUL R4, R5, R6",
    "SUB R7, R1, R8",
    "AND R9, R10, R11",
    "LOAD R12, [100]",
    "OR R13, R9, R14",
    "ADD R15, R16, R17",
    "SUB R18, R19, R20"
]
number_of_instructions = len(instructions)
scalar_cycles = 8
superscalar_cycles = 5
out_of_order_cycles = 4
thread1_instructions = 8
thread2_instructions = 8
smt_total_instructions = thread1_instructions + thread2_instructions
smt_cycles = 9
scalar_ipc = number_of_instructions / scalar_cycles
superscalar_ipc = number_of_instructions / superscalar_cycles
out_of_order_ipc = number_of_instructions / out_of_order_cycles
smt_ipc = smt_total_instructions / smt_cycles
execution_units = 2
scalar_utilisation = (
    number_of_instructions /
    (scalar_cycles * execution_units)
) * 100
superscalar_utilisation = (
    number_of_instructions /
    (superscalar_cycles * execution_units)
) * 100
out_of_order_utilisation = (
    number_of_instructions /
    (out_of_order_cycles * execution_units)
) * 100
smt_utilisation = (
    smt_total_instructions /
    (smt_cycles * execution_units)
) * 100
print("SUPERSCALAR AND SMT SIMULATION\n")
print("Processor Model\tInstructions\tCycles\tIPC\tUtilisation")
print(
    "Scalar\t\t\t",
    number_of_instructions,
    "\t\t\t",
    scalar_cycles,
    "\t\t\t",
    round(scalar_ipc, 2),
    "\t\t\t",
    str(round(scalar_utilisation, 2)) + "%"
)
print(
    "Superscalar\t\t\t"
)

```

Figure 5(a): Python simulation of scalar, superscalar, out-of-order and SMT execution.

```

>>> ===== RESTART: C:/Users/jimin/AppData/Local/Programs/Python/Python313/co3_ca_3.py =====
SUPERSCALAR AND SMT SIMULATION

Processor Model      Instructions    Cycles    IPC    Utilisation
Scalar               8              8         1.0    50.0%
Superscalar          8              5         1.6    80.0%
Out-of-Order         8              4         2.0    100.0%
SMT                  16             9         1.78   88.89%
>>>

```

Figure 5(b): Instructions per cycle and resource-utilization output.

PERFORMANCE ANALYSIS

Processor model	Instructions	Cycles	IPC	Utilization
Scalar	8	8	1.00	50.00%
Superscalar	8	5	1.60	80.00%
Out-of-order	8	4	2.00	100.00%
SMT	16	9	1.78	88.89%

Compared with scalar execution, superscalar issue improves IPC by 60%, and out-of-order scheduling doubles IPC while fully utilizing the two execution units. SMT completes sixteen instructions from two threads in nine cycles; when one thread cannot use an execution slot, the other thread helps keep the units busy. Speculation further supports this parallelism by preventing unresolved branches from immediately emptying the instruction window.

RESULT: Out-of-order execution reaches 2.00 IPC and 100% utilization, while SMT sustains 1.78 IPC and 88.89% utilization across two instruction streams.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation